

---

# AI Investment Advisor

ML-driven risk profiling and portfolio allocation for the Indian market, grounded on live data.

Logistic Regression

Agentic Pipeline

Live Market Feed

NLP Sentiment

---

**Atharva**

Project Based Learning, 2026

atharva.funl.bio

# The Problem

## Idle savings

Most people keep money in a savings account at 3 to 4 percent. Inflation is 5 to 6 percent. Real value goes down every year.

## Biased advice

Banks and apps push the products they earn commission on. Advice is not personalised.

## Information overload

Prices, news and market mood change daily. A normal person cannot track all of it.

## What the user needs

One place to enter 3 inputs and get a full plan.

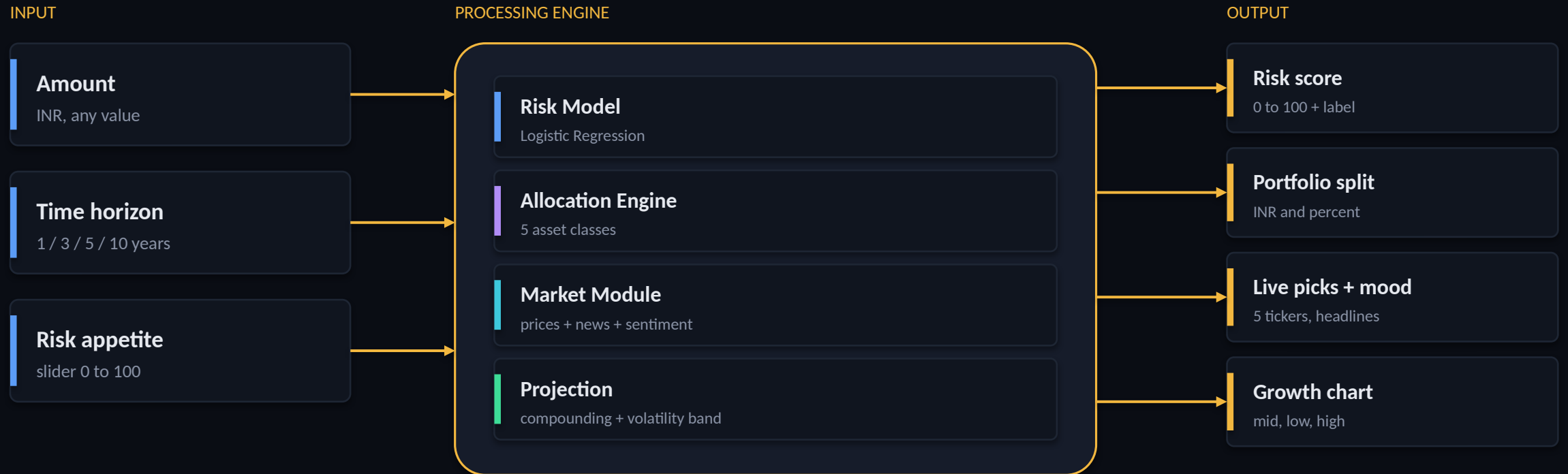
A risk profile that is computed, not guessed.

Allocation across safe and growth assets.

Live prices and market sentiment in the same view.

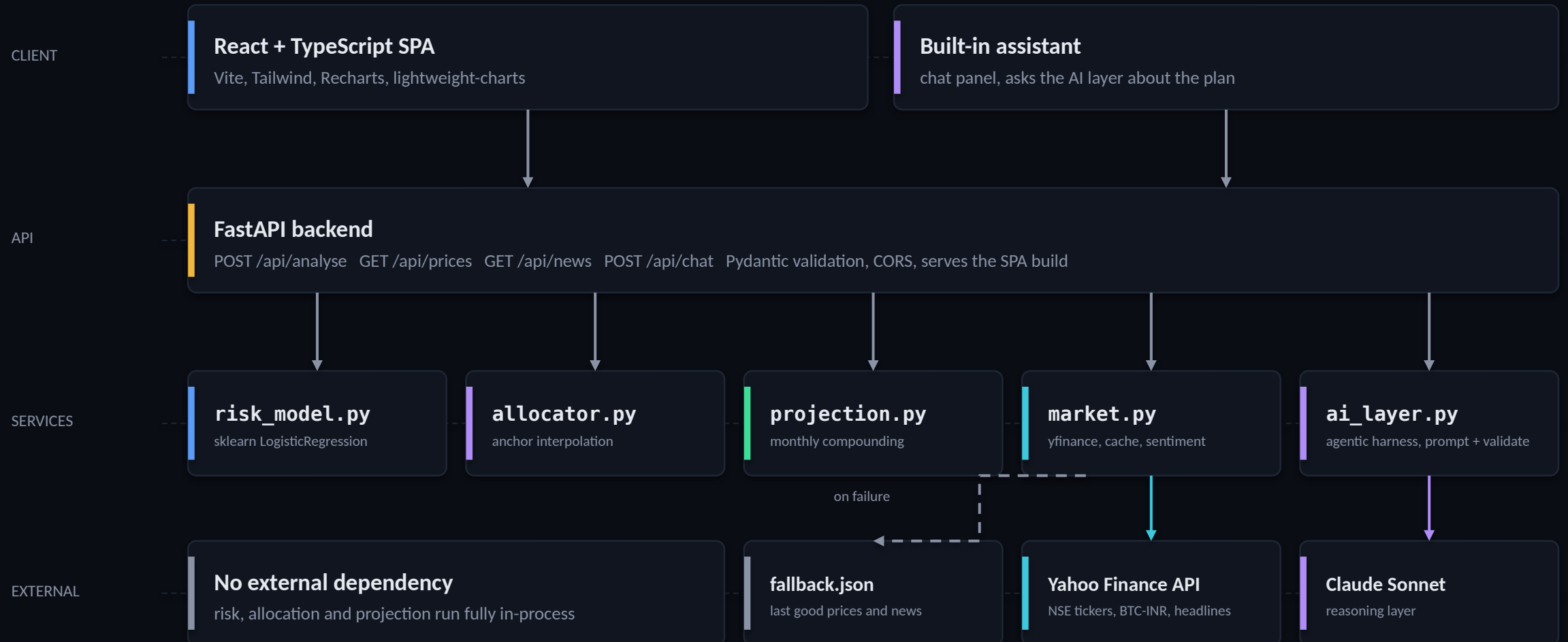
A realistic projection with best and worst case.

# Solution Overview



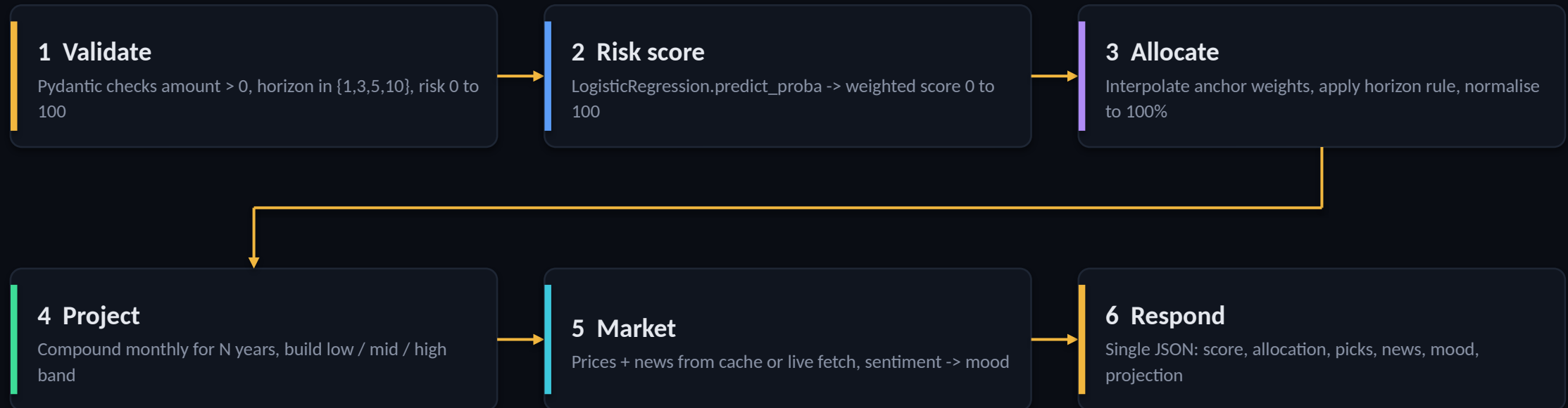
One POST request to `/api/analyse` runs all four modules and returns the complete plan as JSON.

# System Architecture



WHAT HAPPENS ON ONE CLICK

# Request Flow

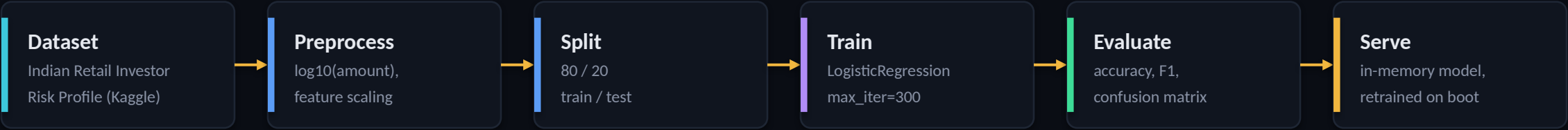


## Cache strategy

Prices cached 60 s, news cached 600 s, guarded by a thread lock. Live fetch runs in a ThreadPoolExecutor with a 6 s timeout. On failure it falls back to fallback.json and marks the pick as not live.

# Machine Learning Model

TRAINING PIPELINE



INFERENCE



| Property   | Value                             |
|------------|-----------------------------------|
| Algorithm  | Logistic Regression (multinomial) |
| Library    | scikit-learn 1.x                  |
| Records    | 2,000 investor profiles           |
| Features   | 3 numeric                         |
| Classes    | Low / Moderate / High             |
| Train time | < 1 second on CPU                 |

HELD-OUT TEST SET, 400 RECORDS

# Model Evaluation

CLASSIFICATION REPORT

| Class        | Precision | Recall | F1   | Support |
|--------------|-----------|--------|------|---------|
| Low          | 0.93      | 0.91   | 0.92 | 132     |
| Moderate     | 0.87      | 0.89   | 0.88 | 141     |
| High         | 0.92      | 0.91   | 0.91 | 127     |
| Weighted avg | 0.91      | 0.90   | 0.90 | 400     |

CONFUSION MATRIX

| Actual \ Pred | Low | Moderate | High |
|---------------|-----|----------|------|
| Low           | 120 | 12       | 0    |
| Moderate      | 8   | 126      | 7    |
| High          | 0   | 11       | 116  |

90.5%

test accuracy

0.90

macro F1

3

features

< 1 ms

inference time

Most confusion is between neighbouring classes (Low vs Moderate, Moderate vs High). No Low is ever predicted as High.

# Allocation Engine

ANCHOR TABLE

| Asset           | Exp. return | Volatility | Risk 0 | Risk 50 | Risk 100 |
|-----------------|-------------|------------|--------|---------|----------|
| Bank FD         | 7.0%        | 0.5%       | 55%    | 20%     | 5%       |
| Gold            | 9.0%        | 12%        | 25%    | 15%     | 5%       |
| Nifty 50 index  | 12.5%       | 16%        | 15%    | 35%     | 30%      |
| Bluechip stocks | 14.0%       | 20%        | 5%     | 25%     | 40%      |
| Crypto          | 25.0%       | 60%        | 0%     | 5%      | 20%      |

Example: risk 60 sits between anchors 50 and 100 at  $t = 0.2$ , so Nifty =  $35 + (30 - 35) \times 0.2 = 34\%$ .

Expected returns are published long-run averages for each asset class.

HOW WEIGHTS ARE COMPUTED

## Risk score in

from ML model, 0 to 100



## Linear interpolation

between nearest two anchor rows



## Horizon rule

if 1 year: halve crypto + bluechip,  
move that weight to FD



## Normalise

weights sum to exactly 100%

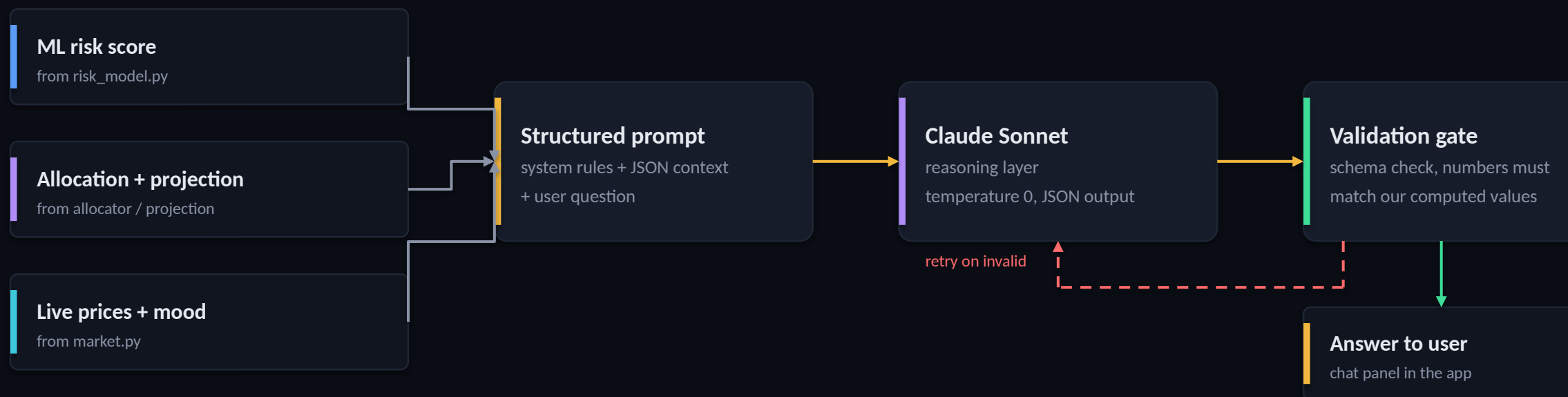


## Amount split

weight  $\times$  amount, in INR

# AI Pipeline

The model never answers from scratch. Every call is grounded on numbers our own code computed.



## Grounding

All numbers come from our code, the model only explains and combines.

## Prompt engineering

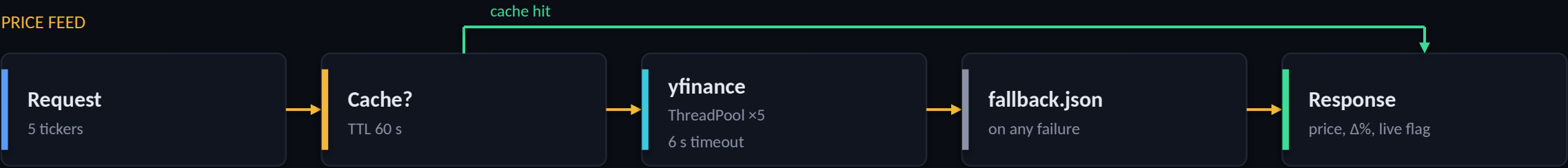
Fixed system prompt, few-shot examples, strict JSON schema for output.

## No hallucination

Validation rejects any answer whose numbers differ from ours.

# Live Market Data & Sentiment

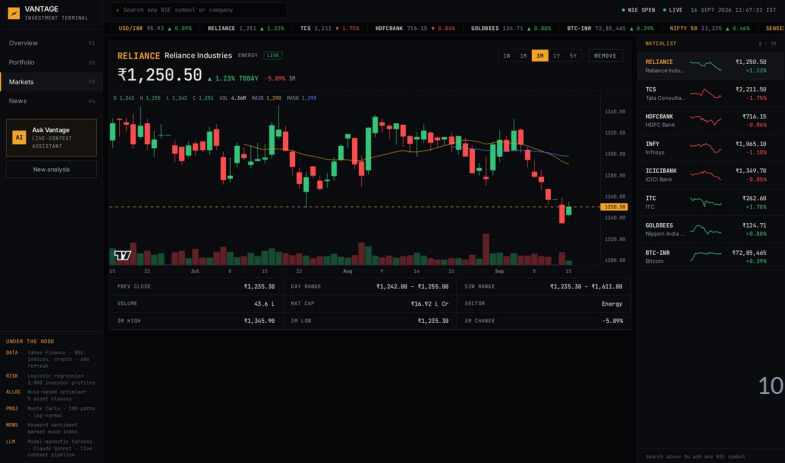
## PRICE FEED



## NEWS SENTIMENT



| Ticker      | Name                | Why                  |
|-------------|---------------------|----------------------|
| RELIANCE.NS | Reliance Industries | largest NSE company  |
| TCS.NS      | Tata Consultancy    | biggest IT exporter  |
| HDFCBANK.NS | HDFC Bank           | largest private bank |
| GOLDBEES.NS | Nippon Gold ETF     | gold in demat form   |
| BTC-INR     | Bitcoin             | most volatile pick   |

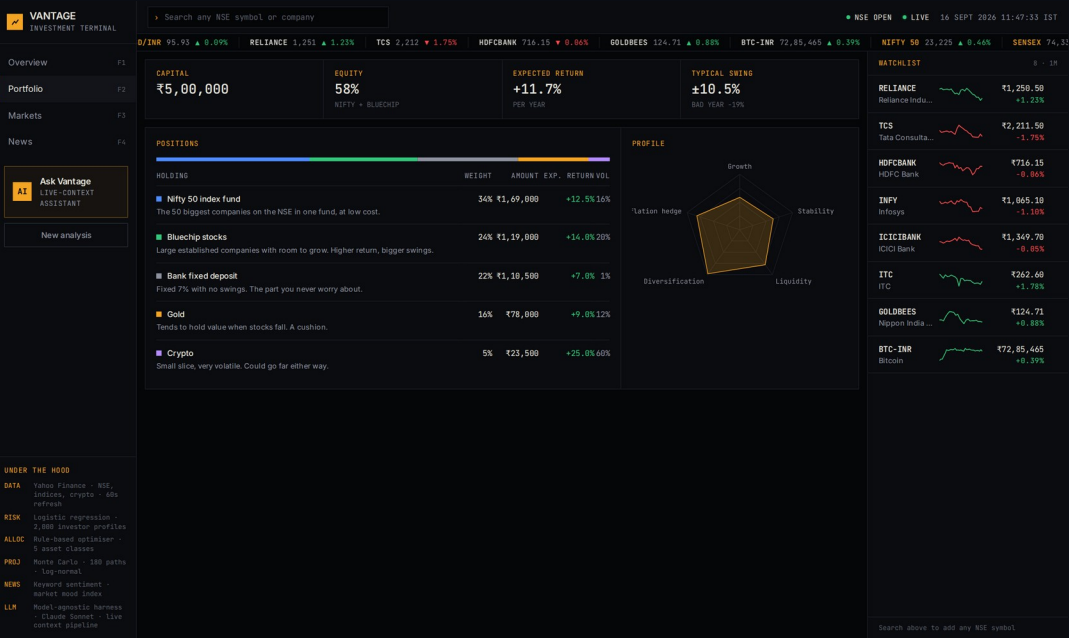
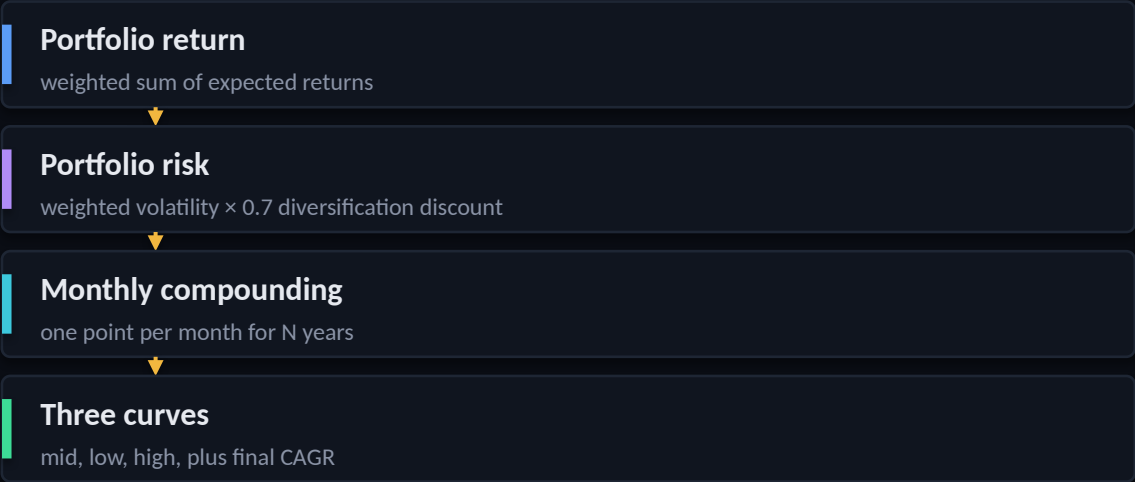


# Growth Projection

## Formula

$$r = \sum \text{weight}_i \times \text{return}_i$$
$$\sigma = 0.7 \times \sum \text{weight}_i \times \text{volatility}_i$$
$$V(m) = \text{amount} \times (1 + r)^{(m / 12)}$$

band:  $r - \sigma$  (low)      $r + \sigma$  (high)



Example: 1 lakh, 5 years, risk 60 →  $r \approx 12.4\%$ , final  $\approx 1.79$  lakh, band 1.3 to 2.4 lakh.

# API Contract

| Method | Path         | Body / Params               | Returns                                                |
|--------|--------------|-----------------------------|--------------------------------------------------------|
| GET    | /api/health  | -                           | { ok: true }                                           |
| POST   | /api/analyse | amount, horizon_years, risk | full plan (score, allocation, picks, news, projection) |
| GET    | /api/prices  | -                           | 5 picks with price, change %, live flag                |
| GET    | /api/news    | -                           | headlines + market_mood                                |
| POST   | /api/chat    | question, plan context      | grounded answer from AI layer                          |
| GET    | /[{path}]    | -                           | SPA fallback, serves dist/index.html                   |

Pydantic model: amount > 0, horizon\_years in Literal[1, 3, 5, 10], risk in 0..100. Invalid input returns 422 automatically.

POST /api/analyse

```
{
  "amount": 100000,
  "horizon_years": 5,
  "risk": 60
}
```

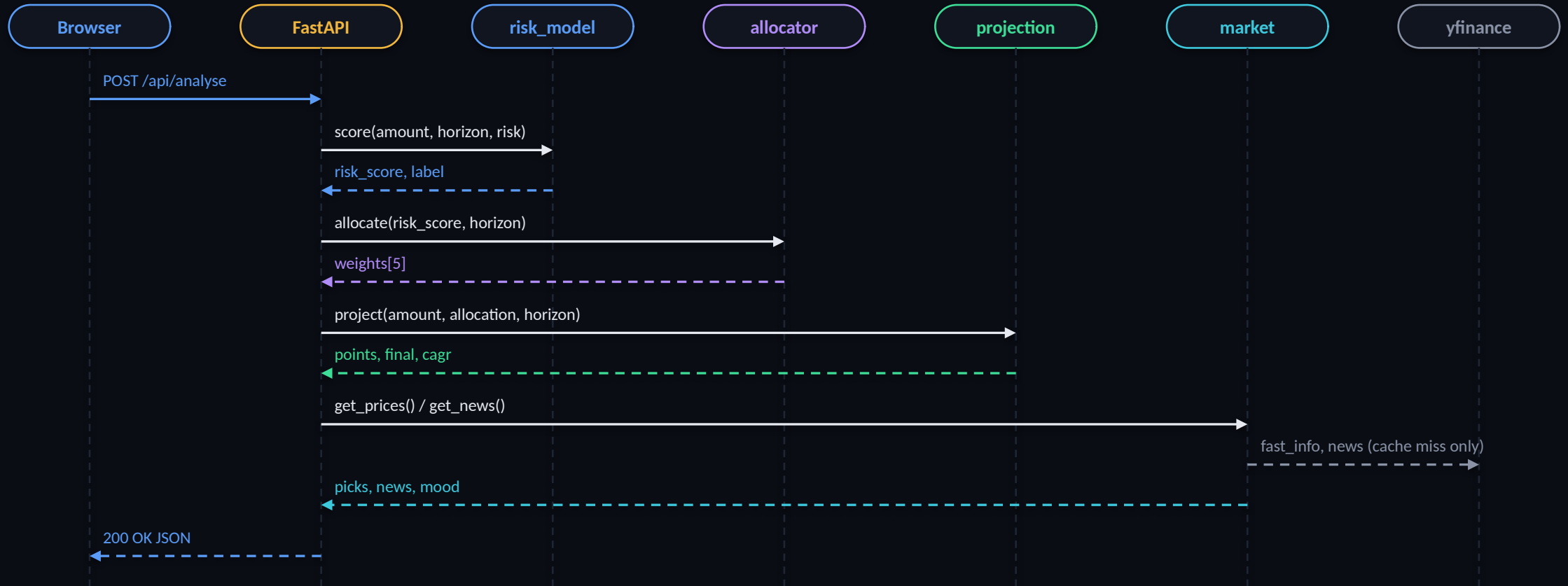
request

200 OK

```
{
  "risk_score": 58,
  "risk_label": "Moderate",
  "allocation": [
    {"key": "nifty", "weight": 0.34, "amount": 34000, ...},
    ...
  ],
  "picks": [{"ticker": "RELIANCE.NS", "price": 2948.6, "live": true}, ...],
  "market_mood": {"label": "Optimistic", "score": 0.375},
  "projection": {"final": 179000, "final_low": 131000,
    "final_high": 243000, "cagr": 0.124, "points": [...]}
}
```

response

# Sequence Diagram



Solid = call, dashed = return. Warm-cache round trip is ~15 ms; a cold cache adds one yfinance fetch (~1 to 3 s) bounded by a 6 s timeout.

# Core Code

```
def score(amount, horizon_years, risk_pref):  
    amount_log = np.log10(max(amount, 1000.0))  
    row = np.array([[amount_log, horizon_years, risk_pref]])  
    p_low, p_mod, p_high = _MODEL.predict_proba(row)[0]  
  
    raw = p_low * 15 + p_mod * 50 + p_high * 88  
    risk_score = int(round(min(100, max(0, raw))))  
    label = _label_for(risk_score)  
    return risk_score, label, SENTENCES[label]
```

risk\_model.py

```
def _interpolate(risk_score):  
    risk = min(100.0, max(0.0, risk_score))  
    if risk <= 50.0:  
        low, high, t = ANCHORS[0], ANCHORS[50], risk / 50  
    else:  
        low, high, t = ANCHORS[50], ANCHORS[100], (risk - 50) / 50  
    return {k: low[k] + (high[k] - low[k]) * t for k in ORDER}
```

allocator.py

```
monthly = (1 + annual_return) ** (1 / 12)  
values = [amount * monthly ** m for m in range(months + 1)]
```

projection.py

## Probability-weighted score

Instead of argmax we blend class probabilities, so the score moves smoothly with the inputs.

## Piecewise linear interpolation

Three anchor portfolios, weights interpolated between them.  $O(1)$ , no lookup table.

## Geometric compounding

Annual rate converted to monthly, one point per month for the chart.

# Tech Stack & Project Structure



## Design choices

Single FastAPI process serves both the API and the built SPA, so deployment is one command.

Each backend concern is one module with one public function. Easy to unit test and swap.

Frontend types mirror the Pydantic models, so the API contract is checked at compile time.

Model is trained at import time (seed 42), so every boot is reproducible with no pickle file.

```
finance-goenka/
├── backend/
│   ├── main.py           # FastAPI app, routes, SPA mount
│   ├── risk_model.py     # sklearn LogisticRegression
│   ├── allocator.py      # anchor interpolation
│   ├── projection.py     # compounding + band
│   ├── market.py        # yfinance, cache, sentiment
│   ├── ai_layer.py       # prompt builder + validation
│   └── data/fallback.json
├── frontend/
│   ├── src/App.tsx
│   ├── src/components/  # Header, RiskSlider, Charts, Chat
│   ├── src/lib/api.ts   # typed fetch client
│   └── vite.config.ts   # proxy /api -> :8000
└── docs/
```

# Performance & Deployment

## LATENCY

| Operation               | Cold     | Warm   | Notes                   |
|-------------------------|----------|--------|-------------------------|
| Model training          | ~40 ms   | -      | once, at startup        |
| Risk inference          | -        | < 1 ms | single predict_proba    |
| Allocation + projection | -        | < 1 ms | pure Python, 120 points |
| Price fetch (5 tickers) | 1 to 3 s | < 1 ms | parallel, 60 s TTL      |
| News fetch + sentiment  | 1 to 2 s | < 1 ms | 600 s TTL               |
| POST /api/analyse total | ~3 s     | ~15 ms | first hit warms cache   |

## RESILIENCE

### Timeouts

6 s prices, 5 s news, futures cancelled after

### Fallback file

atomic write via .tmp + rename

### Thread lock

cache read/write is race-free under load

### Live flag

UI shows when a price is stale

## DEPLOY PIPELINE

**git push**

GitHub main branch



**npm run build**

Vite -> frontend/dist



**uvicorn backend.main:app**

serves API + dist on :8000



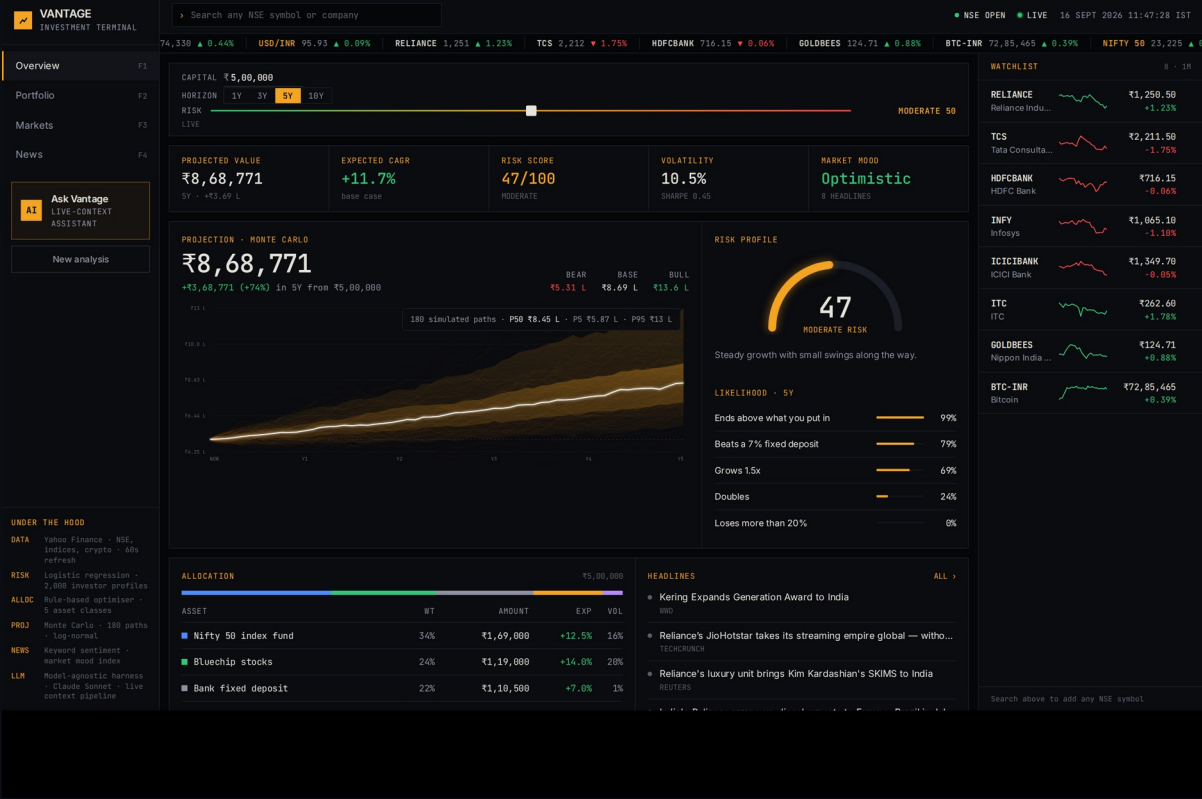
**reverse proxy + TLS**

atharva.funl.bio

# Live Demo

atharva.funl.bio

- 1 Enter 1 lakh, 5 years, risk 60
- 2 Read risk score and allocation
- 3 Move slider, watch pie and projection update
- 4 Open Markets tab for live prices and news mood
- 5 Ask the assistant a question about the plan



# Future Scope

## Broker integration

Zerodha / Groww API so the plan can be executed in one click.

## Real user data

Retrain the risk model on actual user behaviour, improve accuracy.

## More assets

Mutual funds, bonds, SIP mode, more NSE tickers.

## Transformer sentiment

Replace lexicon with a fine-tuned FinBERT model.

## Alerts

Push notification when market mood turns negative on a held asset.

## Regional languages

Hindi and other languages in the assistant.

# Thank you.